

LISBON
DATA SCIENCE
ACADEMY

Analysis and Prediction of Emergency Response Times in San Francisco

San Francisco Fire Department

Report 2

Model Development, Evaluation and Deployment

Prepared for:
San Francisco Department of Emergency Management

Prepared by:
Antonio Quaresma
Data Science Consultant
Impeccable Insights LLC

30/05/2026

Table Of Contents

Table Of Contents	2
1. Summary	3
2. Modelling	4
2.1 Model specifications	4
2.2 Model performance	5
2.3 Alternatives considered	7
3. Model deployment	8
3.1 Deployment specifications	8
3.2 Deployment performance	9
3.3 Unexpected problems	9
3.4 Known issues and risks	10
4. Learnings and future improvements	11
5. Annexes	12

1. Summary

The objective of this project was to develop and deploy a machine learning solution capable of predicting emergency response times for the San Francisco Fire Department using historical dispatch data. The client required a system able to estimate the time between receiving an emergency call and the arrival of the first responding unit on scene, while operating reliably through a production-ready API.

To address this requirement, a supervised regression approach was implemented using engineered operational and temporal features derived from historical emergency call records. Multiple candidate models were evaluated, with an XGBoost Regressor selected as the final solution due to its predictive performance, robustness, and suitability for deployment.

The model was deployed as a FastAPI application hosted on Railway and integrated with a PostgreSQL database for persistent storage of predictions and observed outcomes. The deployed API provides endpoints for both response time prediction and subsequent recording of actual response times, enabling future monitoring of model performance.

During deployment testing and evaluation, the application successfully processed thousands of API requests while maintaining service availability and correctly handling invalid inputs through appropriate validation mechanisms. The solution demonstrated the feasibility of combining machine learning, software engineering, and cloud deployment practices to deliver a practical prediction service aligned with the client's operational requirements.

The final XGBoost model achieved a validation MAE of 219.77 seconds and maintained similar performance on unseen evaluation data.

2. Modelling

2.1 Model specifications

The objective of the modelling process was to predict the response time, measured in seconds, between the reception of an emergency call and the arrival of the first responding unit on scene. Since the target variable is continuous, the problem was formulated as a supervised regression task.

Target Engineering

The target variable was calculated as the time difference between `received_dttm` and `on_scene_dttm`. Because multiple records can be associated with the same incident, a preprocessing step was performed to identify the first responding unit. Following clarification from the client, the modelling target was defined as the response time of the first unit arriving on scene rather than an incident-level aggregated measure.

Records containing missing, invalid, or inconsistent timestamps were removed from the modelling dataset. Negative response times and clear data-quality anomalies were also excluded.

Feature Engineering

Only information available at dispatch time was considered for modelling. Feature engineering focused on extracting operational and temporal characteristics that could influence response times.

Temporal features derived from `received_dttm` included:

- hour of day,
- day of week,
- month,
- weekend indicator.

Additional operational features included variables such as call type, battalion, station area, priority, unit type and supervisor district. Categorical variables were encoded within the modelling pipeline and missing values were handled through imputation or explicit missing categories where appropriate.

Feature selection was performed iteratively using model performance, feature importance analysis, operational interpretability and deployment feasibility as guiding criteria. Variables introducing data leakage or unstable behaviour during validation were excluded from the final feature set.

Model Selection

Several regression algorithms were evaluated during experimentation, including Linear Regression, Random Forest Regressor, Gradient Boosting methods and XGBoost-based models.

Linear models provided a useful baseline but were unable to capture the non-linear relationships present in emergency response operations. Tree-based ensemble methods achieved substantially better performance by modelling interactions between temporal, geographical and operational variables.

The final model selected was an XGBoost Regressor, which provided the best balance between predictive accuracy, robustness across operational subsets, inference speed and deployment simplicity. The model also demonstrated stable performance across different battalions, call types and operational periods.

Hyperparameter Tuning and Evaluation

Hyperparameter tuning was performed through iterative experimentation using chronologically separated training and validation datasets. The process focused on parameters such as learning rate, maximum tree depth, number of estimators, subsampling ratios and regularization settings. Final parameter selection prioritized generalization and stability rather than minimizing a single evaluation metric.

Model performance was evaluated primarily using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). In addition to aggregate metrics, subgroup-level analysis was conducted across battalions, station areas and call types to assess whether the model maintained reasonably levelled performance across different operational contexts.

Final Pipeline

The final modelling pipeline combined preprocessing, feature engineering, categorical encoding and an XGBoost regression model. The trained model and preprocessing artefacts were serialized and integrated into the FastAPI deployment environment, ensuring consistent behaviour between training and production inference.

2.2 Model performance

Model performance was evaluated using both a validation dataset separated from the training data and unseen observations received through the deployed API during the evaluation phase.

Table 1 summarizes the performance of the candidate models evaluated during experimentation.

Table 1. Validation performance of candidate models.

Model	MAE (seconds)	RMSE (seconds)	R ²
Linear Regression	237.97	389.54	0.35
Random Forest	230.64	377.91	0.39
XGBoost	219.77	360.52	0.44

Following deployment, the model was evaluated on unseen observations received through the API. During the final evaluation period, was reported a Mean Absolute Error (MAE) of **230.69 seconds** on the official Friday test dataset. This result is closely aligned with the validation MAE of **219.77 seconds** obtained during model development, suggesting that the model generalized well to previously unseen data without substantial performance degradation.

The XGBoost model achieved the best overall performance across all evaluation metrics and was therefore selected as the final model. Compared with the baseline Linear Regression model, XGBoost reduced the average prediction error by approximately 18 seconds while improving explanatory power.

Subgroup-level analysis was conducted across battalions, neighbourhoods and call-type groups. Although prediction accuracy varied across operational contexts, the model maintained reasonably consistent performance across major battalions and neighbourhoods, although larger differences were observed across call-type groups.

Performance differences were more pronounced across call-type groups. Alarm-related incidents produced the lowest prediction errors, while non-life-threatening incidents and records classified as unknown exhibited higher errors, reflecting their greater response-time variability.

Overall, the selected XGBoost model demonstrated the strongest balance between predictive accuracy, robustness across operational subgroups and suitability for production deployment. The results indicate stable performance across the primary operational contexts represented in the data.

2.3 Alternatives considered

Several regression models were evaluated before selecting the final XGBoost model. The objective was to identify the best balance between predictive performance, robustness and deployment feasibility.

Table 2. Comparison of candidate models and rationale for model selection.

Model	Advantages	Reason Not Selected
Linear Regression	Simple, fast and highly interpretable.	Unable to adequately capture the non-linear relationships present in emergency response operations.
Random Forest	Robust and effective at modelling feature interactions.	Lower predictive performance and slower inference compared with XGBoost.
XGBoost	Strong predictive accuracy, efficient inference and good generalization.	Selected as the final model.

Neural-network-based approaches were considered but not pursued further. Given the structured tabular nature of the dataset, the expected performance gains were unlikely to justify the additional complexity associated with training, tuning, explainability and deployment.

XGBoost was ultimately selected because it provided the best combination of predictive accuracy, robustness across operational contexts and suitability for deployment in a production API environment.

3. Model deployment

3.1 Deployment specifications

The predictive model was deployed as a REST API using **FastAPI** and hosted on **Railway**. The application receives dispatch-time information for a fire unit response, returns a predicted response time in seconds, and stores the prediction in a PostgreSQL database. When actual arrival information becomes available, the stored prediction can be retrieved and compared with the observed response time.

The deployment can be replicated by installing the project dependencies, loading the trained model artifacts, and running the FastAPI application with Uvicorn. In production, Railway automatically starts the service and exposes the API through a public URL. At startup, the application loads the trained preprocessing pipeline and XGBoost model, ensuring that incoming observations undergo the same transformations used during training.

The application consists of four main components:

- **FastAPI application** responsible for request handling and validation.
- **Preprocessing and XGBoost model pipeline** used to generate predictions.
- **PostgreSQL database hosted on Railway** used to store predictions and actual outcomes.
- **Railway deployment infrastructure** responsible for application hosting and database connectivity.

The API exposes two endpoints:

POST /predict_response/

Receives dispatch-time features available when a unit is assigned to an incident, including call type, priority, unit information, location information, and the timestamp at which the call was received. After validation, the observation is processed by the model pipeline, a response time prediction is generated, and the prediction is stored in the database using the pair (unit_id, received_dttm) as a unique identifier.

POST /actual_response/

Receives the unit identifier, call receipt timestamp, and arrival timestamp once the unit reaches the scene. The application retrieves the corresponding prediction from the database, calculates the actual response time as the difference between on_scene_dttm and received_dttm, and returns both the predicted and actual values.

Input validation is performed through FastAPI and Pydantic schemas. Invalid formats, missing required fields, invalid timestamps, unrecognized categorical values, or actual response submissions without a matching prediction return HTTP status code **422**, ensuring data consistency and application reliability.

3.2 Deployment performance

The deployed API was evaluated during the official testing period on 28–29 May 2026. Requests sent before this period were excluded from the analysis because they corresponded to preliminary endpoint verification rather than formal evaluation.

Table 3 summarizes the API performance during the evaluation period.

Table 3. Deployment performance summary for the official API evaluation period (28–29 May 2026), showing successfully processed and failed requests by endpoint.

Endpoint	Successfully Processed	Failed
/predict_response/	6,008	1
/actual_response/	6,001	139
Total	12,009	140

The prediction endpoint achieved a success rate greater than 99.9%, while the actual response endpoint successfully processed the large majority of requests received. Failed requests resulted from validation checks implemented in FastAPI and PostgreSQL persistence logic, ensuring that incorrectly formatted or inconsistent records were rejected rather than stored.

No application crashes, service interruptions, database failures, or deployment rollbacks occurred during the evaluation period. The API remained continuously available and successfully supported the complete prediction and response-time tracking workflow.

Overall, 98.85% of requests received during the official evaluation period were successfully processed.

In addition to deployment reliability, the official evaluation reported a prediction MAE of **230.69 seconds** across 3,004 scored observations, further confirming that the deployed model maintained performance consistent with offline validation results.

3.3 Unexpected problems

Several issues were encountered during deployment that were not apparent during model development.

The most significant challenge was the transition from the local development environment to the Railway production environment. While the application functioned correctly when executed locally, additional configuration was required to ensure proper loading of model artifacts, environment variables, and database connections after deployment.

A second challenge was the migration from a local SQLite database used during development to a PostgreSQL database hosted on Railway. This required modifications to the persistence layer and additional testing to ensure that prediction records could be stored and retrieved reliably across requests.

Unexpected input data was also observed during evaluation. Although the API specification defined a set of dispatch-time features, some requests omitted optional fields or contained combinations of values that were uncommon in the training dataset. To improve robustness, default values and fallback preprocessing logic were implemented, allowing the API to generate predictions even when some non-essential information was missing.

Another observation was that production requests frequently differed from the complete records available during model development. As a result, additional validation and error-handling mechanisms were required to ensure that malformed inputs generated appropriate HTTP 422 responses rather than causing application failures.

Despite these challenges, the application remained stable throughout the evaluation period. No crashes, service interruptions, database outages, or model loading failures were observed after deployment, and no redeployment was required to restore service availability.

3.4 Known issues and risks

Although the deployed application performed reliably during evaluation, several risks remain.

The primary model-related risk is **data drift**. The model was trained using historical observations from 2023–2025 and assumes that future operational patterns remain broadly similar. Changes in staffing levels, station coverage, dispatch procedures, traffic conditions, or call volumes could reduce predictive performance over time. Periodic monitoring and retraining would therefore be required in a production environment.

A second risk is the presence of **previously unseen categorical values**. New unit identifiers, station areas, battalions, neighbourhoods, or call types introduced after model deployment may not be represented in the training data. While the preprocessing pipeline includes mechanisms to handle unknown categories, prediction quality may degrade when such values become frequent.

From an application perspective, the deployment was validated under the request volumes observed during the capstone evaluation period. However, the current architecture has not been stress-tested under substantially higher traffic levels. If deployed as a city-wide operational service, additional monitoring, load testing, and scalability assessments would be recommended.

Finally, the model itself has inherent prediction uncertainty. The best-performing model achieved a mean absolute error of approximately 230 seconds, meaning individual predictions may differ from observed response times by several minutes. Consequently, the model should be viewed as a planning and resource-allocation tool rather than a replacement for operational dispatch decision-making.

For these reasons, both the model and the deployment should be monitored continuously, with periodic performance reviews and retraining as new data becomes available.

4. Learnings and future improvements

This project demonstrated that response times can be predicted with reasonable accuracy using only information available at dispatch time. The analysis also confirmed that response times vary across locations, call types, priorities, and operational districts, supporting the client's hypothesis that service delivery is not uniform across the city.

One important learning is that operational data quality has a direct impact on both analysis and modelling. Missing arrival timestamps, inconsistent records, and location-related anomalies required careful handling throughout the project. Improving data collection processes and validation rules at the source would likely provide greater benefits than incremental improvements to model complexity.

From a modelling perspective, several opportunities for improvement remain. Additional external features such as traffic conditions, weather data, special events, road closures, or population density could potentially improve predictive performance. The current model relies exclusively on dispatch information available in the Fire Department dataset and therefore cannot capture many real-world factors that influence travel time.

Future work could also explore alternative modelling approaches. Although XGBoost produced the best results among the models evaluated, ensemble approaches, geographically-aware models, or separate models for different call categories may yield additional performance gains. Periodic retraining should also be implemented to address data drift and ensure that predictions remain representative of current operating conditions.

From an operational perspective, the current solution predicts response times for individual unit dispatches. A valuable extension would be to develop scenario-planning tools that estimate the impact of station relocations, staffing changes, or increased call volumes on future response times. Such analyses may provide greater strategic value than individual response-time predictions alone.

The fairness analysis also suggests opportunities for further investigation. While much of the variation between neighbourhoods was explained by differences in call mix and priority levels, future work could examine additional socioeconomic, geographic, and infrastructure-related factors to better understand the drivers of service disparities.

Finally, the problem could be reframed from pure prediction to operational optimization. Rather than only estimating future response times, future systems could help identify resource allocation strategies that minimize delays, improve coverage in underserved areas, and support long-term planning decisions for the San Francisco Fire Department.

5. Annexes

5.1 Feature Importance Analysis

The final XGBoost model was interpreted using aggregated feature importance scores. Individual one-hot encoded variables were grouped into their original feature categories to facilitate interpretation.

The results indicate that response time is driven primarily by operational deployment characteristics rather than temporal patterns. Unit-specific information, dispatch priority, and incident classification account for the majority of the model's predictive power, while calendar-related variables contribute relatively little.

This finding is consistent with the exploratory analysis presented in Report 1, which suggested that operational resource allocation and dispatch decisions have a stronger influence on response times than day-to-day seasonal variation.

Table A1. Aggregated feature importance for the final XGBoost model

Feature Group	Importance
unit_id	32%
unit_type	17%
original_priority	15%
call_type_group	13%
call_type	5%
station_area	4%
neighborhood	4%
zipcode	4%
battalion	3%
hour	<1%
other	<1%
month	<1%
day_of_week	<1%

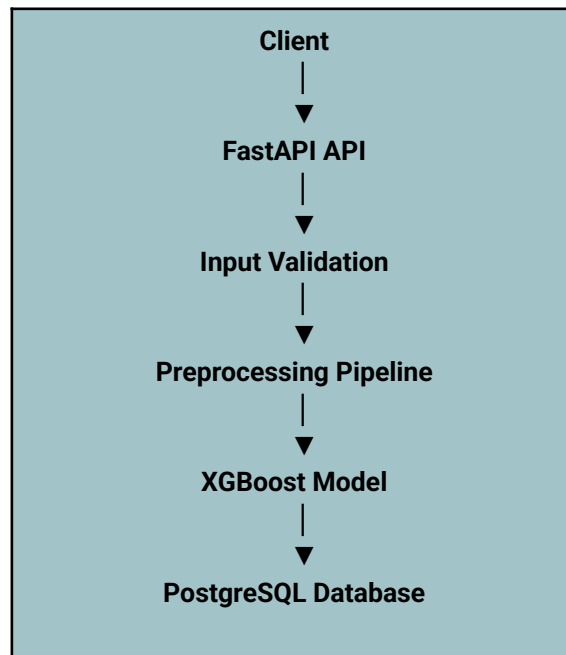
Key operational insight

The strongest predictors of response time are unit characteristics, dispatch priority, and incident classification. Temporal variables such as month, day of week, and hour contribute very little to model performance, suggesting that operational deployment factors are more important drivers of response times than seasonal or weekly patterns.

5.2 API Architecture

The deployed application follows a service-oriented architecture. Incoming requests are validated by FastAPI, transformed using the same preprocessing pipeline employed during model training, scored using the XGBoost model, and stored in a PostgreSQL database hosted on Railway.

Figure A1. High-level architecture of the deployed prediction service.



5.3 Example API Requests and Responses

Figure A2. Example requests and responses supported by the deployed API.

Predict Request

```
{
  "call_type": "Medical Incident",
  "call_type_group": "Potential Life Threatening",
  "original_priority": "3",
  "unit_id": "E01",
  "unit_type": "ENGINE",
  "station_area": "1",
  "battalion": "B01",
  "neighborhood_district": "Financial District/South Beach",
  "zipcode_of_incident": "94105",
  "received_dttm": "2025-01-01T05:10:43"
}
```

Predict Response

```
{
  "unit_id": "E01",
  "received_dttm": "2025-01-01T05:10:43",
  "predicted_response_time_seconds": 644.28
}
```

Actual Response Request

```
{
  "unit_id": "E01",
  "received_dttm": "2025-01-01T05:10:43",
  "on_scene_dttm": "2025-01-01T05:20:43"
}
```

Actual Response Response

```
{
  "unit_id": "E01",
  "received_dttm": "2025-01-01T05:10:43",
  "on_scene_dttm": "2025-01-01T05:20:43",
  "actual_response_time_seconds": 600.0,
  "predicted_response_time_seconds": 644.28
}
```

5.4 – Fairness Assessment Results

To assess whether model performance was consistent across operational and geographic subgroups, prediction errors were evaluated separately for neighbourhoods, battalions, and call type groups.

The results show that model accuracy varies across subgroups, which is expected given differences in incident complexity, operational procedures, and sample sizes. However, for the largest and most operationally relevant groups, performance remains relatively stable, suggesting that the model does not systematically underperform in a specific area of the city.

The official evaluation performed by Friday test set reported a battalion-level MAE standard deviation of **24.39 seconds**, indicating relatively consistent predictive performance across major operational battalions.

The largest errors were generally observed in categories with fewer observations or greater operational variability, while the most common operational groups exhibited error levels close to the overall model MAE.

Table A2. Highest neighbourhood-level prediction errors

Neighbourhood	Records	Mean Response Time (s)	MAE (s)
Presidio	991	814.92	266.51
Treasure Island	724	758.50	266.24
Excelsior	2,567	754.96	241.23
Noe Valley	1,724	598.92	236.87
Financial District/South Beach	8,879	701.34	234.54

Neighbourhoods with the highest model prediction errors. Error variation is relatively moderate despite substantial differences in average response times.

Table A3. Battalion-level prediction performance

Battalion	Records	Mean Response Time (s)	MAE (s)
B05	9,741	599.67	207.99
B02	28,080	611.52	210.99
B04	15,910	585.96	212.25
B01	14,098	628.15	212.43
B06	9,961	629.90	220.51
B07	7,664	661.70	221.25
B08	10,956	707.34	223.06
B09	9,560	720.00	223.94
B03	26,227	684.55	231.08
B10	11,597	691.20	235.97

Battalions AMB, B99 and XXX were excluded from interpretation due to extremely small sample sizes.

Error levels remain relatively consistent across the major operational districts.

Table A4. Call type group prediction performance

Call Type Group	Records	Mean Response Time (s)	MAE (s)
Alarm	32,631	465.20	120.66
Potentially Life-Threatening	70,330	569.85	188.49
Fire	5,976	559.56	209.93
Non Life-threatening	31,677	1,016.34	358.45
Unknown	3,199	733.86	563.61

The model performs best for **Alarm** and **Potentially Life-Threatening** incidents, which represent a large proportion of the operational workload and exhibit more predictable response patterns. Higher errors are observed for **Non Life-threatening** and **Unknown** call categories, likely reflecting greater variability in operational urgency and dispatch behaviour.